

Feasibility of a Software-Factory Approach to Learning Robots That Assemble Additional Robots from Modular Building Blocks

Executive summary

The concept is **feasible in a bounded sense today**, but **not** in the strong science-fiction sense of a robot that fully self-replicates from raw materials. The most realistic near-term interpretation is: a robot learns to assemble **additional robots from pre-manufactured modules** supplied in trays or feeders, inside a structured workcell, using standardized mechanical and electrical interfaces. Research already shows the key ingredients separately: modular self-reconfigurable robotics as a field, autonomous collective construction, robotic LEGO-style assembly from demonstration, and recent long-horizon brick assembly systems that compose learned skills into full builds. What does **not** yet exist as a mature off-the-shelf product is a turnkey stack that combines all of those ingredients with the reliability, safety, and build economics needed for unattended production. ¹

A “software factory” framing is a strong fit for this problem. In current practice, a software factory is a multi-tenant, highly automated assembly plant for software artifacts, built around codified pipelines, Infrastructure as Code, automated testing, release management, and minimal human intervention. For learning systems, that expands naturally into CI/CD plus continuous training and model monitoring. In robotics, ROS build farms, modern simulators, and open robot-learning frameworks already provide most of the raw ingredients for that pipeline. The missing piece is to treat **robot morphology, assembly recipes, skills, data, and safety checks as versioned artifacts** in the same way teams already treat code and containers. ²

The hardware choice is decisive. **LEGO SPIKE/MINDSTORMS and VEX are good learning and proof-of-concept platforms**, but they are not optimized for robot-friendly autonomous assembly because they still rely on friction fits, varied part geometries, and separate wiring. **Cubelets are easier for robots to assemble** because their magnetic faces already carry power and data. **RoFI and RoFICoM-like open-hardware connectors are the most strategically aligned with the goal** because they were designed explicitly for autonomous docking, power/data sharing, self-alignment, and module descriptors. In other words: educational kits are good for learning the problem; purpose-built modular robotics is better for solving it. ³

The highest-probability path is therefore a **hybrid strategy**. Use LEGO/VEX/Cubelets early to validate cell design, perception, and assembly planning. Then migrate quickly to a custom or open modular architecture with **blind-mate power/data, self-aligning connectors, minimal loose cabling, fiducials or embedded pose features**, and **robot-readable module descriptors**. Learning should be used where it helps most — perception, grasp adaptation, error recovery, contact-rich insertion, and long-horizon sequencing — while classical planning, fixtures, and standardized interfaces do the heavy lifting for reliability. Recent work on

LEGO assembly, fingertip vision, and compositional brick-building strongly supports that hybrid approach.

4

Several important dimensions were **not specified** in the request: target robot size, throughput, autonomy threshold, workcell footprint, operating environment, acceptable human intervention, and commercialization geography. Because of that, this report assumes an **indoor tabletop or light industrial workcell, human-supplied module inventory**, and the narrower definition of “build additional robots” as **assembling additional robot units from stocked modules**, not fabricating every component from raw material.

Definitions and scope

A useful working definition of a **software factory** for this problem is: a repeatable, automated production system for robotics software and models that turns a versioned design input into tested, deployable outputs with minimal manual intervention. The DoD Enterprise DevSecOps reference design defines a software factory as a software assembly plant containing multiple pipelines, tools, workflows, environments, and scripts that automate the develop, build, test, release, and delivery lifecycle with minimal human intervention and support multi-tenancy. Google’s MLOps guidance extends that model to ML by adding continuous training, model/data validation, metadata, and automated deployment of the training pipeline itself. 5

For this report, a **learning robot** means a robot whose behavior improves through data rather than only manual reprogramming. That can include imitation learning from human demonstrations, reinforcement learning in simulation or on hardware, perception model training, online adaptation, and hybrid systems in which learned policies are composed with symbolic planning and classical control. Open robot-learning stacks such as LeRobot and large open datasets such as Open X-Embodiment, BridgeData V2, and DROID indicate that the field is increasingly treating robot behavior as a trainable artifact rather than only hand-authored code. 6

The phrase “**autonomously build additional robots**” spans a wide range of ambitions, and the distinction matters. At the low end, it can mean a cell that assembles a second copy of a simple mobile robot from kitted modules once a human has loaded parts into known nests. At the high end, it can imply self-reconfiguration, self-repair, or even open-world self-replication. The literature strongly supports the low and medium interpretations — especially modular docking, collective construction, and kit-based assembly — but does **not** support claiming near-term feasibility for open-world, full-stack self-replication from raw stock. 7

That leads to a practical scope boundary. The strongest near-term target is a **constrained assembly family**: a robot that can assemble one or a few morphologically related robots from a curated module library, automatically identify the resulting configuration, generate or retrieve the right control stack, test the build, and hand the finished robot off for charging or deployment. RoFI’s module descriptors and topology/configuration concepts are especially relevant here because they show how morphology itself can be formalized and exchanged as machine-readable metadata. 8

Evidence from research and the current platform landscape

The academic background is unusually supportive of the core idea. Modular self-reconfigurable robotics has been a recognized research field for decades, with surveys emphasizing versatility, robustness, and shape-changing capability as the main value proposition. More recent state-of-the-art reviews still frame modular self-configurable robots as promising for adaptive automation, but they also make clear that progress is constrained by connector design, control complexity, sensing, and the gap between elegant algorithms and physical hardware. That gap is exactly where a software-factory approach can help: by turning design, simulation, testing, deployment, and failure telemetry into one governed loop. ⁹

On the construction side, TERMES demonstrated autonomous multi-robot three-dimensional construction from specialized passive blocks, with decentralized control and a user-specified desired structure. That result is important because it shows that robots can already execute long-horizon build plans over modular elements without global oversight. What TERMES does **not** solve is general-purpose mechatronic robot assembly with electrical interconnects, software provisioning, and tolerance-sensitive mating across heterogeneous parts. ¹⁰

The most relevant recent evidence comes from **robotic brick assembly**. A 2023 framework showed robotic LEGO assembly and disassembly learned from human demonstration, using a digital twin to validate learned tasks before real deployment on a FANUC arm. In 2025 and 2026, newer work went further: Eye-in-Finger embedded perception at the tool tip to achieve sub-millimeter LEGO manipulation and increased tolerance to calibration error from 0.4 mm to as much as 2.0 mm, while BrickCraft and Prompt-to-Product showed that long-horizon interlocking-brick assembly can be decomposed into reusable skills and linked from specification to real-world build execution. Collectively, this is strong evidence that **bounded autonomous robot assembly from blocks is no longer speculative**. ¹¹

Comparative view of existing modular platforms

The table below compares the most relevant platforms for this use case. The final column is an **analytic assessment for robotic assembly**, not a vendor claim. It is based mainly on connector modality, whether power/data are integrated, presence of loose cables, and whether the platform exposes machine-readable descriptions or open-source internals.

Platform	Typical entry cost	Modularity and connectors	Sensors and actuators	Programmability	Open-source status	Assembly complexity for a robot
LEGO Education SPIKE Prime	\$429.95 for the core set. The set includes 528 elements; the hub has 6 I/O ports, Bluetooth, a speaker, and a 6-axis gyro; Python is supported in the SPIKE app. ¹²	Very high mechanical variety through LEGO Technic/System parts, but power and sensing still route through a hub and cables. ¹³	Motors, force/color/distance options in the SPIKE family; hub-centered architecture. ¹⁴	Scratch-style blocks plus Python. ¹⁵	No open-hardware claim in reviewed LEGO docs. ¹⁶	Medium-high: good large-part manipulability, but snap-fit friction, cable routing, and hub placement make unattended assembly harder.
LEGO MINDSTORMS EV3 / Robot Inventor	Historical MSRP roughly \$349.99 for EV3 and \$359.99 for Robot Inventor; both are retired, and EV3 support is winding down. ¹⁷	Similar LEGO mechanical strengths, with legacy smart-brick architectures. ¹⁸	EV3 included three servo motors and multiple sensors; Robot Inventor used a smart hub and four motors plus sensors. ¹⁸	Icon/block programming; Robot Inventor added Scratch-style coding. ¹⁸	No open-hardware claim in reviewed LEGO docs. ¹⁶	Medium-high for the same reasons as SPIKE, with the extra drawback of retirement.

Platform	Typical entry cost	Modularity and connectors	Sensors and actuators	Programmability	Open-source status	Assembly complexity for a robot
Cubelets Discovery Set	\$225 for the Discovery Set; optional Bluetooth Hat \$43 . ¹⁹	Magnetic faces carry ground, power, and data through three conductors; modules are explicitly Sense/Think/Action blocks. ²⁰	Sensors and actions are modularized at the block level; behavior emerges from arrangement. ²¹	App-controlled behavior changes; Console supports Blockly and C coding. ²²	No open-hardware claim found in reviewed vendor docs. ²³	Low: magnetic self-alignment and integrated power/data are robot-friendly.
VEX IQ Education Kit	\$499.99 for the 2nd-gen Education Kit. The platform has 750+ parts and 4 motors in the education kit; competition kits add more sensors and parts. ²⁴	Strong modular structure with snap-together beams and plates, but still reliant on pins, shafts, and wiring. ²⁵	Motors plus brain/controller and sensor ecosystem such as bumper, optical, distance, and touch LED. ²⁶	Blocks, Python, and C++ via VEXcode. ²⁷	No open-hardware claim found in reviewed VEX docs. ²⁸	Medium: structurally better than Cubelets for real robots, but not self-aligning and still cable-heavy.

Platform	Typical entry cost	Modularity and connectors	Sensors and actuators	Programmability	Open-source status	Assembly complexity for a robot
RoFI platform with RoFICoM	Cost not standardized in the reviewed primary sources; RoFICoM is self-buildable from open hardware sources and a listed BOM. ²⁹	Purpose-built modular robotics platform; genderless, 90-degree symmetric connector with blind docking, integrated power and data , automatic orientation detection, and a reconfiguration-focused morphology. ³⁰	Universal modules expose 3 DoF , IMU, and planned dock-centered ToF sensing; modules use ESP32 control and smart servos. ³¹	Open platform with downloadable hardware and software sources; shared firmware assumptions and module descriptors/configurations. ³²	Explicitly open-source and open-hardware in the reviewed project docs. ³³	High fabrication effort up front, but low docking complexity once standardized.

The headline comparison is straightforward. If the goal is a **demo this quarter**, Cubelets or very large, highly constrained LEGO/VEX builds are the fastest path. If the goal is a **credible autonomous assembly platform**, the connector and module standard should look more like RoFICoM than like a classroom kit. The literature on modular docking and self-aligning connectors points in the same direction: integrated mechanical, power, and data interfaces reduce control burden, error accumulation, and manual rework. ³⁴

Reference hardware and integration architecture

A practical hardware architecture for this project should be optimized around **assembly robustness, not general dexterity**. For a first serious prototype, the recommended cell is a **single table-mounted manipulator with one overhead RGB-D camera, one wrist camera, and fixture-based part presentation**. A second arm only becomes necessary when the build sequence includes bimanual stabilization, frequent regrasping, or simultaneous use of separate seating and fastening tools. The recent LEGO assembly literature suggests that fine correction at the contact interface matters more than brute-force dexterity; the Eye-in-Finger result is particularly relevant because it shows that fingertip sensing can greatly relax global calibration error during tight-tolerance insertion. ³⁵

The manipulator itself does **not** have to be an industrial arm in early phases. Low-cost open arms linked to LeRobot are enough to learn perception, sequencing, and dataset collection, while desktop cobots such as the UFACTORY Lite 6 or DOBOT MG400 are more credible choices once repeatability and runtime matter. The Lite 6 is marketed with ROS/Python/C++ support and a 440 mm reach; the MG400 is a compact desktop collaborative arm with 440 mm reach, collision detection, and stated ± 0.05 mm repeatability in vendor materials. That makes both suitable for a structured assembly cell, although the Lite 6's ecosystem is

stronger for open software integration while the MG400's repeatability spec is more compelling for insertion-heavy work. ³⁶

End-effectors should be **modular** from the start. The minimum set is: a parallel gripper for module bodies, a vacuum tool for flat covers or trays, and a compliance-enhanced fine-insertion tool for connector seating. For LEGO-like tasks, a fingertip camera or “eye-in-finger” style tool is justified surprisingly early because friction contacts and partial occlusion dominate the failure budget. For larger block systems with magnetic or keyed docking, a simpler parallel gripper is usually enough. ³⁷

Vision should be layered. An overhead camera handles scene state, tray occupancy, and gross pose estimation. A wrist camera handles approach refinement and fiducial reading. A fingertip sensor or close camera handles the last few millimeters of insertion. The simulator should mirror all three because perception regressions are one of the easiest ways for embodied systems to “pass CI” but fail in the cell. Isaac Sim explicitly supports robotics simulation, physically based synthetic data generation, and integration of URDF/CAD-derived assets, while Gazebo Harmonic remains a strong open-source baseline for ROS-native validation and sensor simulation. ³⁸

Mechanically, the single biggest improvement is to **eliminate loose wires** from the build vocabulary. RoFICoM is illustrative here: it is a genderless, symmetric connection mechanism that establishes firm mechanical, power, and data connection without prior synchronization, shares power and UART communication, and is explicitly designed for desktop 3D printing. Self-aligning electropermanent-magnet connector research points to the same design pattern more generally: if the connector itself contributes alignment and engagement tolerance, the planner can be simpler and the cell can be faster. ³⁹

That leads to the core integration recommendation:

```
flowchart LR
    subgraph Factory["Software factory"]
        A[Robot family spec and module DSL]
        B[CAD and connector library]
        C[URDF/SDF and assembly recipe generation]
        D[Simulation and digital twin]
        E[Policy training and evaluation]
        F[CI/CD, signing, release, rollback]
    end

    subgraph Cell["Assembly cell"]
        G[Overhead RGB-D and wrist vision]
        H[Planner plus skill library]
        I[Manipulator(s) and tool changer]
        J[Fixtures, nests, trays, feeders]
        K[Finished robot QA and configuration]
    end

    A --> B --> C --> D --> E --> F
    D --> H
```

```
E --> H
F --> H
G --> H --> I
J --> I
I --> K
K -->|telemetry and failures| D
K -->|datasets and test traces| E
```

Tooling should also be intentionally boring. Early cells should prefer **indexed trays, hard stops, V-nests, and simple hold-down fixtures** over flexible conveyors. Conveyors are valuable later for throughput, but they also introduce timing variability, additional sensing demands, and more failure modes. TERMES succeeded partly because its block interface and environment were purpose-built; the same lesson applies here. Use fixtures to collapse uncertainty wherever possible, and spend learning budget on perception and recovery rather than on avoidable variation. ⁴⁰

Software factory and learning stack

The software factory should treat **robot design, assembly plans, learned skills, simulation scenes, and safety constraints as first-class build artifacts**. In practical terms, that means the source repository should hold at least: module CAD and connector specs; module descriptors and topology rules; URDF/SDF or USD assets; skill definitions; demonstration data manifests; test scenes; training configs; deployment recipes; and acceptance tests. The DoD software-factory model and Google's MLOps maturity model both support this framing: code, pipelines, environments, and released artifacts belong in the factory, and ML requires additional automation around data, training, validation, and retraining triggers. ⁵

A strong pipeline for this project would look like this:

1. **Specification:** choose a robot family, module set, and connector version.
2. **Generation:** produce CAD variants, URDF/SDF/USD assets, bill of process, and a nominal assembly graph.
3. **Simulation:** validate kinematics, collision envelopes, perception visibility, and insertion geometry in Gazebo and/or Isaac Sim.
4. **Skill training:** train or fine-tune policies for grasping, local alignment, seating, and recovery.
5. **Verification:** run software tests, simulated task suites, replay-based regressions, and hardware-in-the-loop smoke tests.
6. **Deployment:** package ROS nodes, policies, and assembly recipes; sign and deploy to the workcell.
7. **Runtime data loop:** collect traces, failures, intervention labels, and acceptance-test results back into the training and replay store. ⁴¹

ROS already gives many of the needed software-engineering habits: guided developer workflow, tests, linters, PR validation, and build farms that generate binary packages and run CI across platforms. What robotics often lacks is a **deployment operating system** for combining those practices with behavior regressions, replayed sensor traces, model versioning, and environment drift. DOS^R is relevant here because it proposes exactly that sort of CI/CD architecture for data-driven robots, with environment adaptation, data replay, and performance profiling. A 2025 study on software reconfiguration in robotics is also a reminder that fielded robotic systems need controlled, auditable reconfiguration mechanisms rather than ad hoc updates. ⁴²

Learning methods and what data they really require

For this use case, the best learning architecture is **hybrid**.

Classical planning should own high-level build order, topological constraints, collision checking, and route planning between part nests and the partial build. This is where modular descriptors, assembly graphs, and symbolic preconditions are valuable. Recent long-horizon assembly systems such as BrickCraft effectively use compositional decomposition for that reason: they bridge high-level assembly intent and physical execution by breaking tasks into reusable primitives. ⁴³

Imitation learning is the best starting point for assembly primitives such as pick, orient, align, seat, verify, and recover. Survey work on robotic assembly confirms that learning from demonstration has been especially useful where manipulation features and sequencing can be shown by a teacher. More recent assembly papers also keep coming back to the same pattern: learn from demos first, then improve force adaptation or insertion robustness with RL or online practice. ⁴⁴

Reinforcement learning is most useful in contact-rich stages — especially insertion, compliant seating, and error recovery. The literature on manipulation primitives and insertion shows why: directly learning low-level torques or joint commands is data-hungry and brittle, but learning over **structured primitives** or admittance-control parameters transfers better and generalizes more reliably. That is especially important if the goal is to build many variants of a robot family without retraining the full stack from scratch. ⁴⁵

Perception learning should focus on object pose, fiducials, connector state, and build-state verification. Here the field is moving rapidly toward reusable open datasets and libraries. BridgeData V2 contributes over 60,000 trajectories across 24 environments; DROID contributes 76,000 trajectories or 350 hours gathered across 564 scenes and 84 tasks; Open X-Embodiment assembled data from 22 robots and 527 skills; and LeRobot now provides a unified open library for data collection, storage, streaming, training, and deployment. The main lesson is not that your project needs those exact dataset sizes. It is that **generalist** robot building needs large and diverse data, whereas a **single bounded workcell** can often succeed with far less if geometry is standardized and simulation is good. ⁴⁶

The most honest way to describe data requirements is therefore:

- For a **narrow assembly cell**, limited demonstrations may be enough for skill primitives if the environment is structured, the connector is forgiving, and digital-twin verification is strong. BrickCraft is encouraging evidence here because it shows compositional generalization from a limited set of demonstrations. ⁴⁷
- For **generalization across many structures, module variants, lighting states, and partial-build conditions**, you should expect the data burden to move toward multi-thousand or multi-ten-thousand trajectory regimes, especially for perception. BridgeData V2 and DROID are the relevant scale references. ⁴⁸
- For **cross-embodiment or generalist robot-building**, the field is now in foundation-model territory, where datasets like Open X-Embodiment matter and the software system must support data provenance, replay, and model governance. ⁴⁹

Reliability, safety, economics, and legal considerations

Safety and reliability should be designed in from day one, because robot accidents disproportionately occur during non-routine operations such as setup, maintenance, testing, and adjustment — exactly the states this project will spend a lot of time in. OSHA's robotics overview points to those risks, and the same OSHA resources reference ISO 10218 and ISO/TS 15066 principles for safe robot-system integration and collaborative operation. Functional safety standards such as IEC 61508 become relevant once safety-related control functions and interlocks become part of the architecture. ⁵⁰

The dominant failure modes for this concept are predictable. On the **mechanical** side: mis-grasps, partial insertions, accumulated alignment error, connector wear, damage to pins or magnets, and unstable partial builds. On the **electrical** side: intermittent blind-mate contacts, bus addressing errors, and battery or charging faults. On the **software** side: model regressions, stale calibration, incorrect morphology detection, simulation drift, and broken rollback paths. On the **operational** side: tray depletion, damaged modules, lighting changes, and too many “small” manual exceptions that quietly destroy autonomy. Recent work on fingertip sensing and robotic LEGO assembly strongly implies that insertion recovery and local verification must be explicit parts of the build plan, not afterthoughts. ⁵¹

Maintainability improves sharply if the built robot can **describe itself**. RoFI's descriptor/configuration model is promising here: modules can exchange shape and capability metadata, and the system can distinguish topology from physical configuration. In a software-factory framing, that means a newly built robot should boot into a self-identification routine, publish its module list/topology/firmware versions, receive the matching configuration package, and then run acceptance tests before joining the fleet. ⁸

Prototype cost ranges

The ranges below are **illustrative hardware-and-integration ranges**, not quotes. Because no specific budget or facility assumptions were provided, they exclude major facility changes and usually exclude external training compute. The broad uncertainty comes less from the arm itself than from fixtures, feeders, safety equipment, spare inventory, and engineering labor.

Phase	Objective	Illustrative hardware stack	Estimated BOM range	Why the range is wide
Proof-of-concept	Demonstrate that one robot can assemble a second simple robot or module chain from stocked parts.	One or two low-cost open arms in the SO-ARM100 / SO-101 class; one low-cost RGB-D camera such as OAK-D Lite / OAK-D; trays, lights, fixtures; SPIKE, Cubelets, or custom oversize modules. Current references: SO-ARM100 3D-printed frame \$31.90 and servo kit \$220 ; OAK-D family from \$169–\$329 ; Cubelets Discovery \$225 ; SPIKE Prime \$429.95 . 52	~\$2k–\$8k	Whether you already own a printer/ workstation, whether you use commercial kits or custom modules, and how much fixturing you need.
Functional prototype	Assemble one bounded family of robots reliably, auto-configure them, and pass end-of-line tests.	Desktop cobot such as UFACTORY Lite 6 or DOBOT MG400; parallel and/or vacuum gripper; overhead plus wrist camera; custom nests; safety enclosure; custom module inventory. Current references: Lite 6 around \$3.3k–\$3.5k , gripper \$388.80 , vacuum gripper \$302.40 ; MG400 marketed as a 440 mm desktop cobot with collision detection; OAK 4 D \$849 or lower-cost OAK variants. 53	~\$15k–\$45k	Custom fixtures, safety, module redesign, test instrumentation, and engineering time usually dominate the arm bill.
Pilot	Small pilot cell producing multiple units per week with traceability, QA, spares, and operator procedures.	Dual cobots or arm-plus-gantry, multi-camera perception, feeders or indexed conveyors, QA/test station, safety controls, spare connectors/modules, logging and fleet management. OSHA/ISO integration principles become materially important here. 54	~\$75k–\$250k	System integration, guarding, spare inventory, QA, uptime targets, and staffing. Labor often exceeds hardware cost at this stage.

The key economic conclusion is that **module design is a bigger cost lever than arm selection**. If the parts require loose wire routing, tight blind insertions, or many regrasp steps, you will spend the budget on fixturing, debugging, and operator interventions. If the modules are redesigned for blind-mate power/data, large chamfers, keyed geometry, and obvious verification states, the same arm suddenly looks much more capable. RoFiCoM and self-aligning connector research both support that conclusion. ⁵⁵

Legal, IP, and ethical considerations

From an IP perspective, the biggest issue with LEGO-based approaches is **branding and trademark use**, not just mechanics. LEGO’s Fair Play policy makes clear that the LEGO logo must not be used on unofficial sites and that LEGO trademarks should only be used in limited nominative ways that do not imply sponsorship. If this project ever becomes a product, that means packaging, domains, product naming, and UI language all need careful treatment if LEGO compatibility is part of the story. ⁵⁶

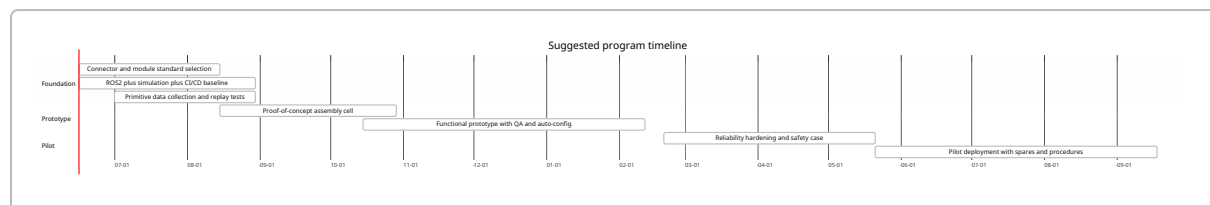
If the long-term strategy is an **open modular platform**, the open-hardware standard matters. OSHWA’s definition requires that the design files be publicly available in the preferred form for modification and that branding/trademarks be kept distinct so downstream users do not imply endorsement. For a project that mixes proprietary kits, open connectors, and learned software, this pushes toward a clean separation between **mechanical interface standard**, **reference implementation**, and **brand**. ⁵⁷

On the regulatory side, if a commercial system with AI-enabled perception or automation is sold in the EU, the **EU AI Act** timeline is relevant: the Act applies progressively, with major transparency and high-risk rules taking effect in stages and the majority of rules coming into force beginning **August 2, 2026**, with full roll-out foreseen by **August 2, 2027**. The exact obligations depend on the final product and market, but the practical message is that logging, transparency, human oversight, and documentation should be designed in now rather than bolted on later. ⁵⁸

Ethically, the main issues are less exotic than the phrase “self-building robot” suggests. The real concerns are: worker safety during setup and maintenance; surveillance or privacy risks from always-on cameras; hidden bias or brittleness in perception and action models; and exaggerated claims about autonomy or “self-replication” that obscure the amount of human-supplied infrastructure involved. A responsible program should keep humans in the loop for exception handling, make intervention boundaries explicit, maintain audit logs, and describe the system accurately as **module-based autonomous assembly**, not unrestricted machine self-replication. ⁵⁹

Roadmap and overall assessment

The highest-probability roadmap is a staged one in which the **module standard is stabilized before the learning stack is allowed to scale**. Teams often do the reverse, and that is where time disappears: they collect data on a part vocabulary that later turns out to be fundamentally hostile to autonomous assembly.



A sensible staffing model for the first 12–18 months is small but cross-functional: one robotics systems lead, one controls/perception engineer, one mechanical engineer focused on modules/connectors/tooling, one ML engineer or robot-learning engineer, and one technician/integration specialist. After the first functional prototype, add safety/compliance support and QA/test engineering. This aligns with the evidence: most failures in this domain sit at the seams between mechanics, perception, control, and deployment, not inside any one discipline. ⁶⁰

A milestone set that is rigorous enough to matter would look like this:

Stage	Exit criterion	Core skills needed	Principal risks
Discovery	Module standard chosen; digital twin running; CI/CD and replay tests in place.	Mechanical design, ROS 2, simulation, DevOps/MLOps.	Choosing a connector vocabulary that looks educationally convenient but is hostile to autonomous assembly.
Proof-of-concept	Robot assembles a second simple robot or module chain with repeatable success in a fixed workcell.	Vision, grasping, assembly planning, fixtures.	Over-reliance on learning when simple fixturing would have solved the problem more robustly.
Functional prototype	End-to-end build of one bounded robot family from stocked modules, with auto-configuration and end-of-line tests.	Controls, contact-rich manipulation, data engineering, software release management.	Calibration drift, connector wear, unbounded exception handling.
Pilot	Measured uptime, operator procedure, spare strategy, versioned releases, rollback, and safety sign-off.	QA, safety engineering, operations, compliance.	Hidden manual work, brittle changeovers, and under-budgeted systems integration.

The bottom-line assessment is:

- **Technically feasible now** if the target is a **bounded module library** and a **structured workcell**.
- **Marginal with LEGO/VEX alone** except as a learning or demonstration platform.
- **Most promising with robot-centered connectors** that integrate mechanics, power, and data.
- **Best implemented as a software factory** that versions morphology, assembly routes, datasets, models, tests, and releases together.
- **Economically justified only if module/interface design is optimized for robotic assembly**, because otherwise exception handling will dominate operations.

Open questions and limitations

The main open questions are not conceptual; they are engineering and product-definition questions:

- What exact robot family is being built: mobile base, manipulator, swarm module, educational rover, or self-reconfigurable lattice robot?

- What level of human support is acceptable: tray reload only, intervention on failures, or lights-out operation?
- Is the long-term goal **education**, **research infrastructure**, or a **commercial pilot line**?
- Will the system need to support only one module generation, or several backward-compatible generations with mixed inventories?
- How much of the control stack must be open, and how much can depend on proprietary vendor firmware?

Until those are fixed, the most robust conclusion is this: **combine a software-factory operating model with a connector-first hardware strategy, and the project is viable; combine the software-factory model with classroom kits alone, and it is likely to remain an impressive demo rather than a dependable autonomous robot-building system.**

1 9 https://www.researchgate.net/publication/3344812_Modular_Self-Reconfigurable_Robot_Systems_Grand_Challenges_of_Robotics
https://www.researchgate.net/publication/3344812_Modular_Self-Reconfigurable_Robot_Systems_Grand_Challenges_of_Robotics

2 5 UNCLASSIFIED
https://dodcio.defense.gov/Portals/0/Documents/DoD%20Enterprise%20DevSecOps%20Reference%20Design%20v1.0_Public%20Release.pdf?utm_source=chatgpt.com

3 12 13 SPIKE™ Prime – STEAM Set - Grades 6 - 8 | LEGO® Education
https://education.lego.com/en-us/products/lego-education-spike-prime-set/45678?utm_source=chatgpt.com

4 11 35 60 <https://arxiv.org/abs/2305.15667>
<https://arxiv.org/abs/2305.15667>

6 <https://arxiv.org/abs/2602.22818>
<https://arxiv.org/abs/2602.22818>

7 10 40 <https://dash.harvard.edu/entities/publication/73120378-c973-6bd4-e053-0100007fdf3b>
<https://dash.harvard.edu/entities/publication/73120378-c973-6bd4-e053-0100007fdf3b>

8 <https://rofi.fi.muni.cz/platform/descriptors/>
<https://rofi.fi.muni.cz/platform/descriptors/>

14 SPIKE Prime Support | Everything You Need | LEGO® Education
https://education.lego.com/en-us/product-resources/spike-prime/troubleshooting/spike-prime-faqs/?utm_source=chatgpt.com

15 SPIKE Prime Support | Everything You Need | LEGO® Education
https://education.lego.com/en-us/product-resources/spike-prime/downloads/python/?utm_source=chatgpt.com

16 <https://www.lego.com/en-us/legal/notices-and-policies>
<https://www.lego.com/en-us/legal/notices-and-policies>

17 18 <https://www.lego.com/en-us/product/lego-mindstorms-ev3-31313>
<https://www.lego.com/en-us/product/lego-mindstorms-ev3-31313>

19 Cubelets Discovery Set - Modular Robotics | Cubelets robot blocks
https://modrobotics.com/cubelets-discovery-set/?utm_source=chatgpt.com

20 21 <https://modrobotics.com/cubelets-getting-started-guide/>
<https://modrobotics.com/cubelets-getting-started-guide/>

- 22 <https://modrobotics.com/cubelets-apps/cubelets-app/>
<https://modrobotics.com/cubelets-apps/cubelets-app/>
- 23 Cubelets robot blocks - Modular Robotics | Cubelets robot blocks
https://modrobotics.com/?utm_source=chatgpt.com
- 24 25 VEX IQ - VEX Robotics
https://www.vexrobotics.com/iq?utm_source=chatgpt.com
- 26 IQ Competition Kits - VEX Robotics
https://www.vexrobotics.com/iq-competition-kits.html?utm_source=chatgpt.com
- 27 VEX Robotics | Install VEXcode for VEX IQ | Setup Guide - VEX Robotics
https://www.vexrobotics.com/vexcode/install/iq?utm_source=chatgpt.com
- 28 VEX Robotics | VEXcode | Coding for VEX Robotics Robots - VEX Robotics
https://www.vexrobotics.com/vexcode?utm_source=chatgpt.com
- 29 30 34 39 55 <https://rofi.fi.muni.cz/hardware/connector/>
<https://rofi.fi.muni.cz/hardware/connector/>
- 31 <https://rofi.fi.muni.cz/hardware/universal-module/>
<https://rofi.fi.muni.cz/hardware/universal-module/>
- 32 33 <https://rofi.fi.muni.cz/>
<https://rofi.fi.muni.cz/>
- 36 53 <https://www.robotshop.com/products/ufactory-6-axis-robot-arm-lite-6>
<https://www.robotshop.com/products/ufactory-6-axis-robot-arm-lite-6>
- 37 51 <https://arxiv.org/abs/2503.06848>
<https://arxiv.org/abs/2503.06848>
- 38 41 <https://developer.nvidia.com/isaac/sim>
<https://developer.nvidia.com/isaac/sim>
- 42 <https://docs.ros.org/en/foxy/The-ROS2-Project/Contributing/Developer-Guide.html>
<https://docs.ros.org/en/foxy/The-ROS2-Project/Contributing/Developer-Guide.html>
- 43 47 <https://arxiv.org/abs/2605.07605>
<https://arxiv.org/abs/2605.07605>
- 44 <https://www.mdpi.com/2218-6581/7/2/17>
<https://www.mdpi.com/2218-6581/7/2/17>
- 45 <https://arxiv.org/abs/2011.00778>
<https://arxiv.org/abs/2011.00778>
- 46 48 <https://arxiv.org/abs/2308.12952>
<https://arxiv.org/abs/2308.12952>
- 49 <https://arxiv.org/abs/2310.08864>
<https://arxiv.org/abs/2310.08864>
- 50 54 59 <https://www.osha.gov/robotics>
<https://www.osha.gov/robotics>

⁵² <https://www.seeedstudio.com/SO-ARM100-3D-printed-Enclosure-p-6409.html>
<https://www.seeedstudio.com/SO-ARM100-3D-printed-Enclosure-p-6409.html>

⁵⁶ <https://www.lego.com/en-us/legal/notices-and-policies/fair-play>
<https://www.lego.com/en-us/legal/notices-and-policies/fair-play>

⁵⁷ <https://oshwa.org/definition/>
<https://oshwa.org/definition/>

⁵⁸ <https://ai-act-service-desk.ec.europa.eu/en/ai-act/timeline/timeline-implementation-eu-ai-act>
<https://ai-act-service-desk.ec.europa.eu/en/ai-act/timeline/timeline-implementation-eu-ai-act>